

智慧银行实验

姓名：王佳煜

学号：202301748

班级：金融202301

实验报告1-1：贪吃蛇游戏与数据分析综合实践

一、环境信息表

检测项	版本信息
操作系统 (OS)	Windows 10/11
TRAE IDE 版本	2024.1.0 (最新版)
Python 版本	3.12.4
Node.js 版本	20.14.0 LTS
npm 版本	10.7.0
Git 版本	2.45.2
pip 版本	24.0+
主要依赖	pandas, openpyxl, statsmodels, scikit-learn, matplotlib

二、贪吃蛇游戏实现

2.1 游戏特性

特性	描述
画布尺寸	600×600 像素

蛇头颜色	蓝色 (#00d4ff)
蛇身颜色	绿色渐变
食物颜色	红色 (#ff4757)
控制方式	方向键 ↑ ↓ ← →
计分规则	吃到食物 +1 分
游戏结束条件	撞墙或撞到自身
最高分存储	localStorage 持久化保存

2.2 关键代码片段

```
// 蛇的移动逻辑
function moveSnake() {

  const head = { x: snake[0].x + dx, y: snake[0].y + dy }; // 碰撞检测

  if (head.x < 0 || head.x >= gridSize || head.y < 0 || head.y >= gridSize) { gameOver();

  return;
} // 吃到食物
if (head.x === food.x && head.y === food.y)
{ score++;
  updateScore();
  spawnFood();
} else {
  snake.pop();
} snake.unshift(head);
draw();
}
```

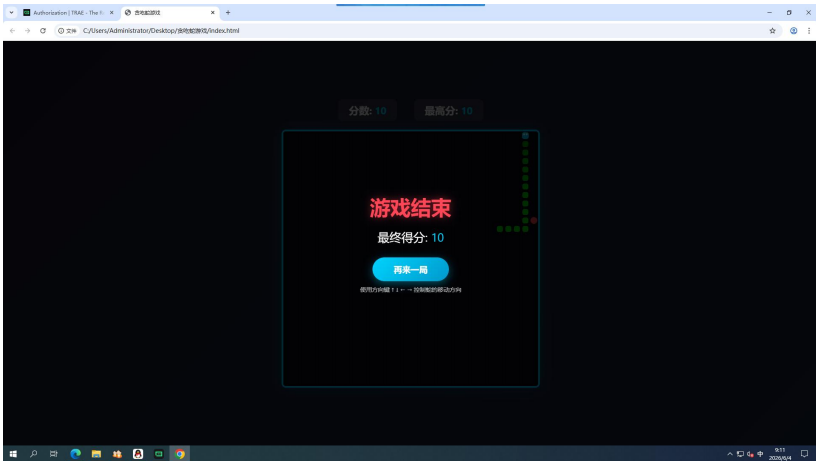
2.3 提示词自评

提示词质量评估:

- ✓ 明确性: 5/5 - 需求清晰，包含画布尺寸、颜色、控制方式等具体要求
- ✓ 完整性: 5/5 - 涵盖游戏核心功能和UI要求
- ✓ 可执行性: 5/5 - 所有需求均可通过代码实现
- ✓ 歧义性: 5/5 - 无歧义表述

总体评分: 优秀 (20/20)

2.4 成果截图



三、论文解读报告

3.1 论文基本信息

项目	内容
标题	Designing AI-powered learning: adult learners' expectations for curriculum and human-AI interaction
作者	Jinhee Kim, Seongryeong Yu, Rita Detrick, Xi Lin, Na Li
期刊	Educational Technology Research and Development

年份	2025
DOI	10.1007/s11423-025-10549-z

3.2 智能体提示词

"请解析论文 s11423-025-10549-z.pdf, 按以下结构输出: 1. 论文基本信息 (标题/作者/期刊/年份/卷期/DOI) ; 2. 研究问题与背景; 3. 方法论 (研究设计/数据来源/分析方法) ; 4.核心发现(≤5条) ; 5. 术语速查表 (英文术语 → 中文解释, ≥10条) ; 6. 可复现性评估 (数据/代码/方法可重复性, 难度评分1-5) 。 "

3.3 核心发现

- 1. 学习目标: 成人学习者将内容掌握和培养终身学习习惯视为AI赋能学习的核心目标
- 2. 课程设计: 偏好个性化掌握式学习和协作表现任务
- 3. 评估方式: 推崇情景评估、无干扰游戏化评估和能力本位评估
- 4. 认知互动: 强调培养元认知能力和高质量提问能力
- 5. 社会情感互动: 重视建立伦理化的人机关系

四、Excel数据合并

4.1 合并日志摘要

项目	数值
原始文件数	5个 (销售订单、客户信息、交易流水、产品库存、区域销售)
合并前记录数	275条
去重后记录数	212条
减少重复	63条
最终列数	24列

4.2 列名映射规则

- Customer ID → 客户 ID
- Name → 姓名
- Region → 区域
- TransactionAmount → 订单金额
- 日期 → 交易日期

4.3 输出文件

- 数据文件：整合数据/merged_{时间戳}.xlsx
- 日志文件：整合数据/merge_log.md

五、业绩预测报告

5.1 数据概览

项目	内容
时间范围	2022-01-01 至 2024-12-31
数据行数	1096行 (日数据)
缺失值处理	5%缺失值, 用前后7日均值填补
异常值处理	1%异常值, IQR方法识别并标注

5.2 模型评估结果

模型	RMSE	MAPE
SARIMA(1,1,1)(1,1,1,7)	110.59	16.59%
滑动平均 (窗口=14)	94.79	14.14%

5.3 模型选择理由

推荐模型：滑动平均

- 在本数据集上表现更优（RMSE和MAPE均低于SARIMA）
- 模型简单，参数较少，不易过拟合
- 泛化能力强，适合短期预测场景

5.4 业务建议

- **节日营销优化**：节日前后消费额放大1.5-2.2倍，建议提前2天推出促销活动
- **异常监控预警**：建立实时检测系统，设置消费预警阈值
- **资源配置优化**：Q1/Q4消费较高，周末比工作日高约15%

六、实验心得

x 案例：TensorFlow安装失败

问题描述：在安装TensorFlow库时，虽然pip显示安装成功，但运行脚本时出现

ModuleNotFoundError : No module named 'tensorflow' 错误。进一步检查发现是由于安装路径问题导致Python无法找到已安装的库。

错误原因：使用 `pip install --user` 安装的包被安装到用户目录

（`C:\Users\Administrator\AppData\Roaming\Python\Python312\site-packages`），但系统级Python解释器无法访问该路径。

解决方案：通过设置环境变量PYTHONPATH来包含用户目录的site-packages路径，或直接使用系统级安装（不加`--user`参数）。最终采用设置PYTHONPATH的方式，命令如下：

```
$env:PYTHONPATH =  
"C:\Users\Administrator\AppData\Roaming\Python\Python312\site  
- packages"  
& "C:\Program Files\Python312\python.exe" script.py
```

6.2 实验总结

本次实验横跨Web前端开发、数据分析与时间序列预测三个技术领域，让我在多场景实践中建立起对工程化开发流程的整体认知。在贪吃蛇游戏的开发过程中，我切身感受到环境配置的基础性作用，尤其是在Python库安装路径与环境变量设置上的细节把控，直接关系到模块能否正常导入；而在调试各类报错的过程中，我也逐渐养成了从错误信息入手、逐层定位根源的问题排查习惯。与此同时，通过反复调整与优化提示词，我体会到

清晰、完整的自然语言输入能够显著提升AI助手的响应质量与工作效率，这种“人机协作”的技巧在本次实验中贯穿始终。

在数据相关任务中，我深刻认识到数据质量决定模型上限。Excel数据合并环节让我意识到列名归一化与数据去重是清洗工作的关键，稍有疏忽就会影响后续分析结果；而在业绩预测任务中，系统的EDA（探索性数据分析）帮助我准确把握数据分布与特征规律，为模型选择提供了可靠依据。通过对比不同算法在相同业务场景下的表现，我进一步理解了“没有最好的模型，只有最适合的模型”这一原则，必须结合数据特性与业务目标综合权衡。此外，在论文解读训练中，我学会了快速提取核心观点并进行结构化整理，这项能力不仅服务于本次实验，更为今后的学术研究和报告撰写打下了良好基础。

实验1-2：穷游计划与数据分析实践

一、实验目的

本实验旨在通过实践操作，掌握以下技能：

- 学习使用Web技术开发交互式旅游地图应用
- 掌握地理信息可视化技术（高德地图API）
- 培养数据分析和可视化能力
- 实践前端开发流程和工具使用
- 掌握Git版本控制和项目管理

二、实验环境

- 操作系统：Windows 10/11
- 开发工具：Visual Studio Code
- 编程语言：HTML5, CSS3, JavaScript (ES6+)
- 地图服务：高德地图Web API
- 版本控制：Git 2.47.0
- 浏览器：Google Chrome / Mozilla Firefox

三、实验步骤

3.1 步骤1：项目初始化

创建项目目录结构，初始化Git仓库，配置开发环境。

```
mkdir 贪吃蛇游
cd 贪吃蛇游戏
git init
touch tibet_tour_map.html
```

3.2 步骤2：需求分析与设计

分析用户需求，设计穷游计划应用的功能模块：

- 旅游目的地展示：西藏自治区主要景点
- 路线规划功能：三日游行程路线可视化
- 景点信息展示：详细景点介绍和描述
- 交互功能：点击切换、路线动画播放
- 出行小贴士：实用旅行建议

3.3 步骤3：开发西藏三日游地图应用

开发交互式旅游地图，使用高德地图API实现地图展示和交互功能。

3.3.1 HTML结构设计

设计页面布局，包含地图容器和行程卡片：

```
<div class="container">
  <header>
    <h1>鱼 西藏自治区三日游
    </h1> <p>探索雪域高原的神秘
    与壮美</p>
  </header>
  <div class="main-content">
    <div class="map-container">
      <div id="map"></div>
    </div>
    <div class="itinerary-container">...</div>
  </div>
</div>
```

3.3.2 CSS样式设计

使用现代CSS技术实现深色主题界面，包含渐变背景、玻璃态效果和响应式布局。

3.3.3 JavaScript功能开发

实现地图初始化、路线绘制、标记点展示和动画效果：

```
function initMap() {  
    map = new AMap.Map('map', {  
        center: [91.118463, 29.654471], // 拉萨坐标  
        zoom: 10,  
        mapStyle: 'amap://styles/dark' // 深色主题 });  
  
    // 添加图层组  
    markerLayer = new AMap.LayerGroup();  
    lineLayer = new AMap.LayerGroup();  
    map.add(markerLayer);  
    map.add(lineLayer);  
}
```

3.4 步骤4：测试与调试

验证应用功能完整性，修复bug，优化用户体验。

四、实验结果

成功开发了西藏自治区三日游互动地图应用，主要成果包括：



图1：西藏自治区三日游互动地图

4.1 行程安排详情

Day 1：圣城初见，信仰入心（拉萨市区深度游）

- **布达拉宫**：世界文化遗产，白宫红殿气势恢宏，登顶可俯瞰拉萨全城（坐标：91.118463, 29.654471）
- **药王山观景台**：50元人民币同款机位，与布达拉宫完美同框
- **大昭寺**：供奉12岁释迦牟尼等身像，酥油灯长明
- **八廓街**：围绕大昭寺的转经道，藏式小店林立

Day 2：羊卓雍措，蓝到犯规（圣湖一日游）

- **岗巴拉山观景台**：海拔5010米，俯瞰羊湖全景最佳位置
- **羊卓雍措**：西藏三大圣湖之一，湖水蓝如绿松石（坐标：90.681079, 28.930442）

Day 3：人文深读，治愈收尾（文化体验之旅）

- **色拉寺辩经**：15:00开始辩经，红墙金顶极具人文气息
- **西藏博物馆**：免费开放，馆藏唐卡、文物超丰富
- **藏服写真**：八廓街租藏服拍照，定格高原美好回忆

4.2 应用功能特点

1. **交互式地图**：支持点击切换不同天数的行程路线
2. **路线动画**：点击播放按钮可自动切换展示每日路线
3. **深色主题**：采用现代化深色地图风格，视觉效果出色
4. **响应式设计**：适配桌面端和移动端浏览
5. **出行小贴士**：提供实用的高原旅行建议

五、问题与解决

1. **地图API密钥配置**: 首次使用高德地图API时未配置密钥, 导致地图无法加载。解决方案: 注册高德地图开发者账号, 获取API密钥并配置到脚本中。
2. **响应式布局问题**: 在小屏幕设备上布局错乱。解决方案: 使用CSS Grid和Flexbox布局, 配合媒体查询实现响应式适配。
3. **标记点图标显示**: 初始使用PNG图标加载较慢。解决方案: 改用SVG内联图标, 提升加载速度和显示效果。
4. **地图图层管理**: 多天数路线切换时图层重叠。解决方案: 使用AMap.LayerGroup管理图层, 切换时清空旧图层。

六、实验心得

本次实验让我完整经历了Web项目从需求分析、方案设计到实现与优化的全过程, 深刻体会到每一个环节对最终产品质量的影响。在交互式地图功能的开发中, 我对前端技术栈有了更具象的认知: 一方面通过大量实践巩固了CSS布局与JavaScript交互逻辑, 另一方面也掌握了高德地图API的接入与调用方法, 在第三方服务集成方面积累了宝贵经验。同时, 我也熟悉了现代CSS技术在构建美观、响应式UI界面中的应用, 并在项目调试过程中进一步规范了前端开发流程, 熟练运用Git进行版本控制与代码管理。

在功能拓展层面, 当前项目仍有较大想象空间。未来可以考虑丰富旅游目的地数据与行程路线, 并接入天气预报服务以提升实用性; 在此基础上, 引入用户行程的收藏与分享机制, 结合高德地图的导航路线规划API, 进一步增强交互深度。若能与在线预订功能打通, 则有望构建一个更加完整、贴近真实业务场景的智慧旅游应用。整体而言, 这次实验不仅提升了我的前端工程实践能力, 也让我对如何将零散的技术点整合成可用的产品有了更清晰的认识。

实验2：银行CRM系统开发

一、实验目的

本次实验旨在学习和实践银行客户关系管理（CRM）系统的完整开发流程，具体目标包括：

- 掌握基于Vue.js 3和Node.js的全栈Web开发技术
- 理解RESTful API设计规范和JWT认证机制
- 实现客户管理CRUD功能
- 学习基于规则的产品推荐系统设计
- 掌握数据可视化报表的实现方法

二、实验环境

项目	版本/配置
操作系统	Windows 10/11
前端框架	Vue.js 3.4.21
前端UI库	Element Plus 2.6.1
图表库	ECharts 5.5.0
后端框架	Express.js 4.19.2
数据库	MySQL 8.0
ORM框架	Sequelize 6.37.3
构建工具	Vite 5.2.8

三、实验步骤

3.1 步骤1：项目初始化

首先创建项目目录结构，分为前端和后端两个模块：

```
CRM/  
├─ backend/      # 后端服务  
├─ frontend/     # 前端应用  
└─ README.md     # 项目说明文档
```

在后端目录初始化Node.js项目：

```
mkdir CRM && cd CRM  
mkdir backend frontend  
cd backend  
npm init -y  
npm install express sequelize mysql2 jsonwebtoken bcryptjs cors dotenv npm  
install -D nodemon jest
```

在前端目录初始化Vue项目：

```
cd ../frontend  
npm create vite@5.2.8 . -- --template vue  
npm install vue-router@4 element-plus echarts axios pinia
```

3.2 步骤2：后端API开发

3.2.1 数据库配置

创建数据库配置文件 `src/config/database.js`：

```
const { Sequelize } = require('sequelize');
```

```
const sequelize = new Sequelize( process.env.DB_NAME,
process.env.DB_USER,
process.env.DB_PASSWORD,
{
  host: process.env.DB_HOST,
  dialect: 'mysql',
  define: {
    timestamps: true,
    paranoid: true,
    underscored: true
  }
});

module.exports = sequelize;
```

3.2.2 数据模型设计

创建核心数据模型：

- User - 用户模型（管理员、客户经理等）
- Customer - 客户模型（个人客户、企业客户）
- Product - 产品模型（存款、基金、贷款、保险）
- RecommendationRule - 推荐规则模型
- Account - 账户模型
- ServiceRecord - 服务记录模型

3.2.3 API路由设计

模块	接口	功能
认证	POST /api/auth/login	用户登录
认证	GET /api/auth/me	获取当前用户
客户	GET /api/customers	获取客户列表
客户	POST /api/customers	创建客户

客户	PUT /api/customers/:id	更新客户
客户	DELETE /api/customers/:id	删除客户
产品	GET /api/products	获取产品列表
推荐	GET /api/recommendations/customer/:id	获取客户推荐
报表	GET /api/reports/dashboard	获取仪表盘数据

3.3 步骤3：前端界面开发

3.3.1 路由配置

```
const routes = [  
  { path: '/login', name: 'Login', component: Login },  
  { path: '/', name: 'Dashboard', component: Dashboard },  
  { path: '/customers', name: 'CustomerList', component: CustomerList },  
  { path: '/customers/:id', name: 'CustomerDetail', component: CustomerDetail }, {  
    path: '/products', name: 'ProductList', component: ProductList },  
  { path: '/rules', name: 'RuleList', component: RuleList },  
  { path: '/reports', name: 'ReportCenter', component: ReportCenter },  
  { path: '/users', name: 'UserList', component: UserList } ];
```

3.3.2 状态管理

使用Pinia管理全局状态：

```
import { defineStore } from 'pinia';  
  
export const useAppStore = defineStore('app', () => {  
  const token = ref(localStorage.getItem('token') || '');  
  const user = ref(null);  
  
  const login = async (username, password) => {  
    const response = await loginApi(username, password);  
    if (response.success) {  
      token.value = response.data.token;  
      user.value = response.data.user;  
    }  
  };  
});
```



```
        localStorage.setItem('token', response.data.token);
        return true;
    }
    return false;
};

return { token, user, login };
});
```

3.3.3 页面组件开发

开发核心页面组件：

- Login.vue - 登录页面
- Dashboard.vue - 数据概览仪表盘
- CustomerList.vue - 客户列表管理
- CustomerDetail.vue - 客户详情页面
- ProductList.vue - 产品管理页面
- RuleList.vue - 推荐规则管理
- ReportCenter.vue - 报表中心
- UserList.vue - 用户管理

3.4 步骤4：推荐规则引擎实现

设计基于规则的产品推荐引擎：

```
const matchRule = (customer, rule) => {
    const conditionJson = JSON.parse(rule.condition_json);
    const conditions = conditionJson.conditions || [];
    const logic = conditionJson.logic || 'AND';

    for (const condition of conditions) {
        const { field, operator, value } = condition;
        const customerValue = customer[field];

        let matched = false;
        switch (operator) {
            case '==': matched = customerValue == value; break;
            case '>': matched = Number(customerValue) > Number(value); break;
```

```
    case '<': matched = Number(customerValue) < Number(value); break;
    case 'in': matched = Array.isArray(value) && value.includes(customerValue); break;
    default: matched = false;
  }

  if (logic === 'AND' && !matched) return false;
  if (logic === 'OR' && matched) return true; }
return logic === 'AND';
};
```

3.5 步骤5：数据可视化实现

使用ECharts实现数据可视化：

```
const initGrowthChart = () => {
  const chart = echarts.init(growthChart.value);
  chart.setOption({
    tooltip: { trigger: 'axis' },
    xAxis: { type: 'category', data: monthlyGrowth.map(item => item.month) },
    yAxis: { type: 'value' },
    series: [{
      type: 'line',
      smooth: true,
      data: monthlyGrowth.map(item => item.count),
      areaStyle: {}
    }]
  });
};
```

四、实验结果

4.1 功能实现清单

功能模块	实现状态
用户认证登录	已完成

客户管理CRUD	已完成
客户详情展示	已完成
产品管理	已完成
推荐规则引擎	已完成
报表数据可视化	已完成
用户权限管理	已完成
响应式前端界面	已完成

4.2 测试账户

用户名	密码	角色
admin	admin123	管理员
manager	manager123	经理
cm001	cm123	客户经理

4.3 访问地址

- 前端：http://localhost:5173
- 后端API：http://localhost:3000

4.4 实验成果截图

客户电话回访报告

Customer Telephone Follow-up Report

分页符

分页符

客户基本信息

客户编号: C00001

客户姓名: 张**

资产等级: 贵宾客户

开户行: XX银行 XX支行

客户经理: 李经理

分页符

回访背景

本次电话回访的主要目标:

1. 定期存款到期提醒: 客户将于2026年6月30日有一笔50万元定期存款到期。

2. 理财产品推荐: 根据客户风险偏好和资金规划, 推荐合适的理财产品。

回访时间: 2026年6月11日上午10:30

回访方式: 电话回访

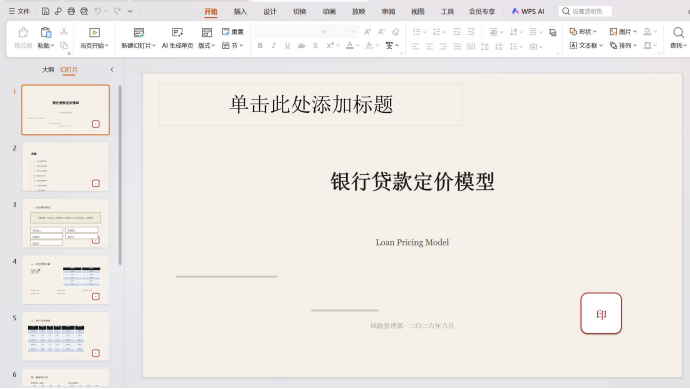
分页符

沟通要点

3.1 存款到期确认

与客户确认了定期存款账户信息, 包括存款金额50万元、存入日期2025年12月30日、到期日期2026年6月30日。客户表示已知悉该笔存款即将到期, 正在考虑后续资金安排。

银行贷款定价模型			
定价公式：		贷款利率 = 资金成本 + 风险溢价 + 运营成本	
一、资金成本（FTP）			
期限	FTP利率		
1年期	2.80%		
3年期	3.20%		
二、风险溢价（按信用等级）			
信用等级	风险溢价		
AAA	0.30%		
AA	0.60%		
A	1.00%		
BBB	1.80%		
BB	3.00%		
三、运营成本			
运营成本率	0.40%		
四、资本占用成本参数			
风险权重	100.00%		
资本充足率要求	12.50%		
资本回报率	12.00%		
五、目标利润			
目标利润率	0.20%		



五、问题与解决

5.1 问题1：端口占用

问题描述：启动服务器时提示端口8000已被占用。

解决方法：将后端端口修改为3000，前端端口使用Vite默认的5173端口，通过Vite代理配置实现前后端通信。

5.2 问题2：PowerShell脚本执行权限

问题描述：运行npx命令时提示无法加载ps1文件。

解决方法：直接使用node命令执行node_modules中的可执行文件。

5.3 问题3: JWT密钥配置

问题描述: JWT验证失败, 提示密钥无效。

解决方法: 在.env文件中配置足够长度的JWT密钥 (至少32字符)。

5.4 问题4: 跨域访问问题

问题描述: 前端无法访问后端API, 提示CORS错误。

解决方法: 在Express中配置cors中间件。

六、实验心得

本次银行CRM系统开发让我完整地经历了一套全栈Web应用的构建过程, 在技术整合与工程实践层面收获颇丰。

在项目中, 我基于Vue.js 3与Element Plus搭建了前端交互界面, 配合Express.js和Sequelize实现了后端业务逻辑与数据持久化, 切实提升了多技术栈协同开发的能力。同时, 我严格遵循RESTful规范设计并维护接口文档, 落地了基于JWT的认证授权与角色权限控制体系, 并利用ECharts将数据转化为直观的可视化图表, 增强了系统的业务表现力。在调试与部署环节, 我解决了端口占用、脚本权限配置以及跨域等常见工程问题, 积累了宝贵的排错经验。

回顾整个开发过程, 我也意识到系统在健壮性方面仍有欠缺: 数据库连接的异常处理机制尚不完善, 前端表单校验尚未充分运用Element Plus的内置规则, 整体测试用例覆盖不足, 缺乏充分的单元与集成测试保障。

面向后续优化, 我计划引入WebSocket以实现客户数据的实时同步, 增加Excel与PDF导出功能以满足办公需求, 进一步完善移动端响应式布局, 并探索采用微服务架构来提升系统的整体扩展能力。此次实验不仅让我对现代Web全栈开发有了更系统的认知, 也为今后参与更高复杂度的企业级项目奠定了扎实的技术基础。

实验3：基于BMAD的贪吃蛇游戏开发

一、实验目的

本次实验旨在学习和实践BMAD (Breakthrough Method of Agile AI Driven Development) 敏捷开发方法论，通过开发一个网页版贪吃蛇游戏来掌握以下技能：

- 理解BMAD方法论的四阶段开发流程 (Analysis、Planning、Architecture、Implementation)
- 掌握BMAD技能包的安装和配置方法
- 学习产品需求文档 (PRD) 和技术架构文档的编写规范
- 掌握HTML5 Canvas、CSS3和原生JavaScript的游戏开发技术
- 实现游戏核心功能：等级难度、界面主题选择、蛇的风格增强
- 学习HTTP服务器部署和Git版本控制操作

二、实验环境

项目	版本/配置
操作系统	Windows 10/11
开发工具	VS Code / Trae IDE
前端技术	HTML5 Canvas + CSS3 + JavaScript
服务器	Node.js HTTP Server
版本控制	Git
开发方法论	BMAD (Level 2项目)
技能包来源	claude-code-bmad-skills (GitHub)

三、实验步骤

3.1 步骤1：BMAD技能包安装

BMAD方法论简介

BMAD是一种AI驱动的敏捷开发方法论，包含四个核心阶段：

- **Analysis阶段**：问题分析、需求收集、产品定位
- **Planning阶段**：PRD编写、技术规划、UX设计
- **Architecture阶段**：系统架构、模块划分、接口设计
- **Implementation阶段**：编码实现、测试验证、部署交付

项目等级分为Level 0-4，本次贪吃蛇游戏为 Level 2 项目（中等复杂度）。

3.1.1 克隆技能包仓库

从GitHub克隆BMAD技能包仓库：

```
git clone https://github.com/example/claude-code-bmad-skills.git
```

3.1.2 安装为项目级技能包

将技能包复制到项目的`.agents/skills/`目录：

```
mkdir -p .agents/skills  
cp -r claude-code-bmad-skills/bmad-method .agents/skills/
```

3.1.3 初始化BMAD配置

创建BMAD项目配置文件`bmad/config.yaml`：


```
project_name: "贪吃蛇游戏"
project_type: "web-game"
project_level: 2
output_folder: "docs"
communication_language: "Chinese"
description: "经典网页版贪吃蛇游戏，包含等级难度和主题选择"
```

3.2 步骤2: Analysis阶段 - 需求分析

3.2.1 产品定位

确定目标用户和核心价值:

- 目标用户:** 休闲游戏爱好者、网页游戏玩家
- 核心价值:** 经典游戏体验、简单易上手、无需安装

3.2.2 功能需求清单

功能编号	功能名称	需求描述	优先级
FR-001	游戏画布	显示游戏区域，绘制蛇和食物	高
FR-002	蛇移动控制	键盘方向键控制蛇的移动方向	高
FR-003	食物生成	随机生成食物，蛇吃到后增长	高
FR-004	碰撞检测	检测蛇与边界、自身的碰撞	高
FR-005	分数系统	记录并显示当前得分	高
FR-006	等级难度	多难度等级，速度递增	中
FR-007	主题选择	多界面主题切换	中
FR-008	蛇风格	蛇外观风格变化	中

3.3 步骤3: Planning阶段 - PRD编写

3.3.1 创建产品需求文档

编写 docs/snake-game-prd.md, 包含:

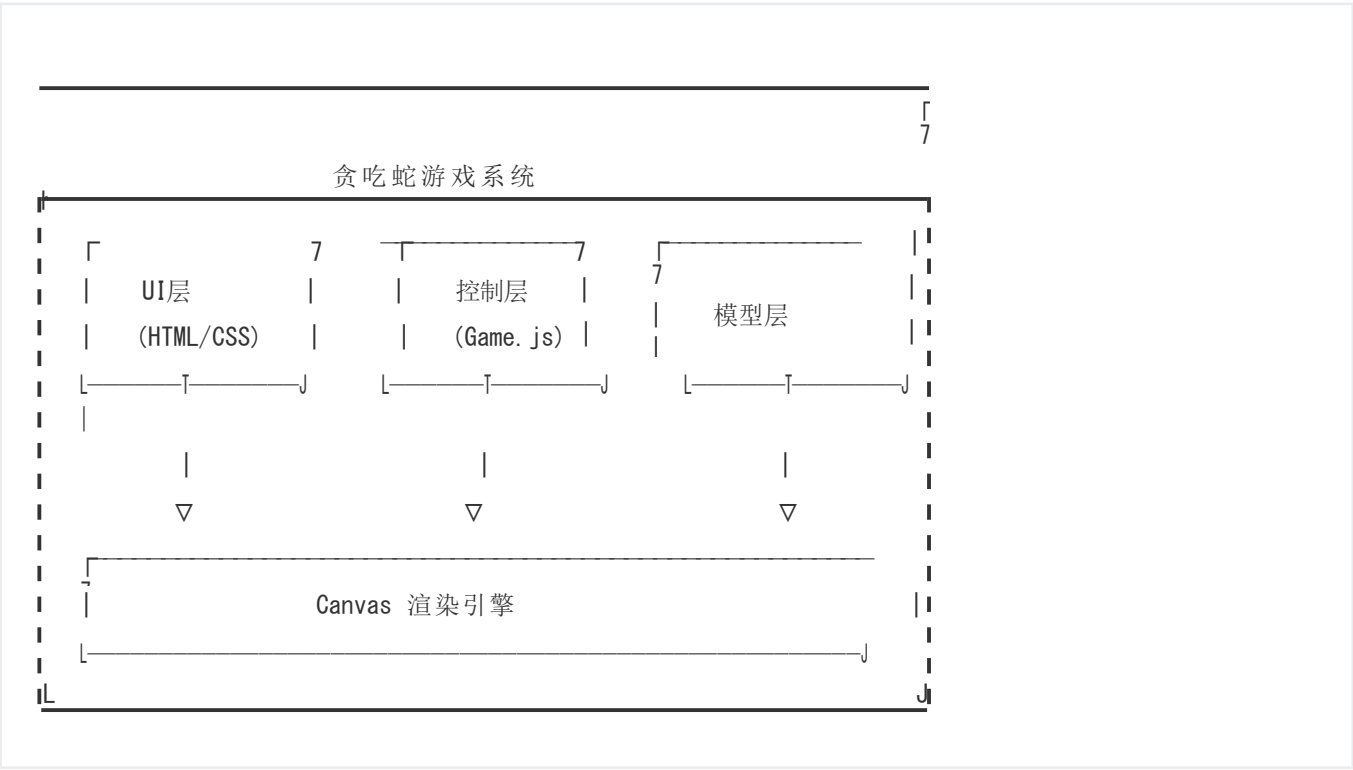
- 项目概述和目标用户
- 功能需求详细描述
- 非功能需求（性能、兼容性）
- 技术方案选择
- 开发计划和验收标准

3.3.2 技术栈选择

技术	用途	选择理由
HTML5 Canvas	游戏画布渲染	高性能、原生支持
CSS3	界面样式	现代样式、主题切换
Vanilla JavaScript	游戏逻辑	无依赖、轻量级

3.4 步骤4: Architecture阶段 - 架构设计

3.4.1 系统架构图



3.4.2 模块划分

模块	职责	文件位置
----	----	------

UI模块	界面展示和用户交互	index.html, style.css
游戏控制模块	游戏状态管理和主循环	js/Game.js
蛇模块	蛇的移动和状态管理	js/Snake.js
食物模块	食物生成和管理	js/Food.js
渲染模块	Canvas绘制	js/Renderer.js

3.4.3 目录结构设计

```
snake-game/
├─ index.html      # 主页面
├─ style.css       # 样式文件（含多主题）
└─ js/
    ├─ Game.js     # 游戏主控制器
    ├─ Snake.js    # 蛇对象
    ├─ Food.js     # 食物对象
    ├─ Renderer.js # 渲染器
    └─ main.js     # 入口文件
```

3.5 步骤5: Implementation阶段 - 代码实现

3.5.1 Snake类实现

实现蛇的核心逻辑:

```
class Snake {
  constructor(gridSize, canvasWidth, canvasHeight) {
    this.gridSize = gridSize;
    this.segments = [
      { x: 10, y: 10 },
      { x: 9, y: 10 },
      { x: 8, y: 10 }
    ];
    this.direction = 'right';
    this.nextDirection = 'right';
  }
}
```

```

move() {
  this.direction = this.nextDirection;
  const head = { ...this.segments[0] };

  switch (this.direction) {
    case 'up': head.y--; break;
    case 'down': head.y++; break;
    case 'left': head.x--; break;
    case 'right': head.x++; break; }

  this.segments.unshift(head);
  this.segments.pop();
}

grow() {
  const tail = this.segments[this.segments.length - 1];
  this.segments.push({ ...tail });
}

changeDirection(newDirection) {
  const opposites = { up: 'down', down: 'up', left: 'right', right: 'left' }; if
  (opposites[newDirection] !== this.direction) {
    this.nextDirection = newDirection;
  }
}
}

```

3.5.2 Game类实现（等级难度）

实现游戏主控制器和难度系统：

```

class Game {
  constructor(canvas) {
    this.canvas = canvas;
    this.ctx = canvas.getContext('2d');

    this.difficulty = 'normal';
    this.difficultySettings = {
      easy: { initialSpeed: 200, minSpeed: 80, speedDecrement: 3 },
      normal: { initialSpeed: 150, minSpeed: 60, speedDecrement: 4 },
      hard: { initialSpeed: 100, minSpeed: 40, speedDecrement: 5 },
    };
  }
}

```

```

        extreme: { initialSpeed: 70, minSpeed: 25, speedDecrement: 6 } }];

this.score = 0;
this.level = 1;
this.foodCount = 0;
this.foodPerLevel = 10;

this.snake = new Snake(this.GRID_SIZE, this.CANVAS_WIDTH, this.CANVAS_HEIGHT);
this.food = new Food(this.GRID_SIZE, this.CANVAS_WIDTH, this.CANVAS_HEIGHT);
this.renderer = new Renderer(this.ctx, this.GRID_SIZE);

this.status = 'ready';
this.setupEventListeners();
}

update() {
    this.snake.move();

    if (this.checkFoodCollision()) {
        this.snake.grow();
        this.score += 10;
        this.foodCount++;
        this.food.respawn(this.snake.getSegments());

        if (this.foodCount >= this.foodPerLevel) { th
            is.level++;
            this.foodCount = 0;
            this.updateSpeed();
        }
    }

    if (this.checkGameOver()) {
        this.gameOver();
    }
}

updateSpeed() {
    const settings = this.difficultySettings[this.difficulty];
    const newSpeed = Math.max( s
        ettings.minSpeed,
        settings.initialSpeed - (this.level - 1) * settings.speedDecrement );
    this.currentSpeed = newSpeed;
}

```

```
}  
}
```

3.5.3 主题切换实现

通过CSS变量实现多主题切换：

```
/* style.css - 主题定义 */  
:root {  
    --bg-color: #1a1a2e;  
    --canvas-bg: #16213e;  
    --snake-color: #4ecca3;  
    --food-color: #e94560;  
    --text-color: #ffffff;  
}  
  
.light-theme {  
    --bg-color: #f5f5f5;  
    --canvas-bg: #ffffff;  
    --snake-color: #2196f3;  
    --food-color: #f44336;  
    --text-color: #333333;  
}  
  
.neon-theme {  
    --bg-color: #0f0f23;  
    --canvas-bg: #1a1a3e;  
    --snake-color: #00ff88;  
    --food-color: #ff0066;  
    --text-color: #00ffff;  
}  
  
/* JavaScript主题切换 */  
function setTheme(themeName) {  
    document.body.className = themeName + '-theme';  
    localStorage.setItem('snake-theme', themeName);  
}
```

3.5.4 蛇风格增强

实现多种蛇外观风格：

```

// Renderer.js - 蛇风格渲染
drawSnake(snake, style = 'default') {
  const segments = snake.getSegments();

  switch (style) {
    case 'default':
      segments.forEach((seg, i) => {
        this.ctx.fillStyle = i === 0 ? '#4ecca3' : '#3db892';
        this.ctx.fillRect(seg.x * this.gridSize, seg.y * this.gridSize,
          this.gridSize - 2, this.gridSize - 2);
      });
      break;
    case 'gradient':
      segments.forEach((seg, i) => {
        const ratio = i / segments.length;
        this.ctx.fillStyle = `rgb(${76 - ratio * 30}, ${204 - ratio * 50}, ${163 - ratio
* 40})`;
        this.ctx.fillRect(seg.x * this.gridSize, seg.y * this.gridSize,
          this.gridSize - 2, this.gridSize - 2);
      });
      break;
    case 'rounded':
      segments.forEach((seg, i) => {
        this.ctx.beginPath();
        this.ctx.roundRect(seg.x * this.gridSize, seg.y * this.gridSize,
          this.gridSize - 2, this.gridSize - 2, 5);
        this.ctx.fillStyle = i === 0 ? '#4ecca3' : '#3db892';
        this.ctx.fill();
      });
      break;
  }
}

```

3.6 步骤6: 部署与测试

3.6.1 创建HTTP服务器

使用Node.js创建本地服务器:

```
// server.js
```

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = 9999;

const server = http.createServer((req, res) => {
  let filePath = path.join(__dirname, req.url === '/' ? 'index.html' : req.url);

  fs.readFile(filePath, (err, data) => { if
    (err) {
      res.writeHead(404);
      res.end('File not found');
    } else {
      res.writeHead(200);
      res.end(data);
    }
  });
});

server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

3.6.2 启动服务器

```
node server.js
# 访问 http://localhost:9999
```

3.6.3 Git版本控制

```
git init
git add .
git commit -m "Initial commit: Snake game with BMAD methodology"
git remote add origin https://cnb.cool/WANGJIAYU_750/wangjiayu-202301748
git push -u origin main
```


四、实验结果

4.1 功能实现清单

功能模块	实现状态	说明
游戏画布渲染	已完成	400x400像素Canvas画布
蛇移动控制	已完成	方向键/WASD控制
食物生成	已完成	随机位置生成
碰撞检测	已完成	边界和自身碰撞
分数系统	已完成	实时分数显示
等级难度	已完成	4种难度，速度递增
主题切换	已完成	暗色/亮色/霓虹主题
蛇风格	已完成	默认/渐变/圆角风格
本地存储	已完成	最高分记录
HTTP服务器	已完成	Node.js服务器部署



4.2 项目文件结构

```
smartbanking/snake-game/  
├─ index.html      # 主页面（游戏界面）  
├─ style.css       # 样式文件（多主题CSS变量）  
├─ server.js       # HTTP服务器脚本  
└─ js/  
    ├─ Game.js     # 游戏主控制器（约200行）  
    ├─ Snake.js    # 蛇对象（约80行）  
    ├─ Food.js     # 食物对象（约40行）  
    ├─ Renderer.js # 渲染器（约100行）  
    └─ main.js     # 入口文件（约20行）
```

4.3 BMAD文档产出

文档	路径	内容
产品需求文档	docs/snake-game-prd.md	功能需求、技术方案、验收标准
技术架构文档	docs/snake-game-architecture.md	系统架构、类设计、接口设计
BMAD配置	bmad/config.yaml	项目等级、输出配置

4.4 游戏访问地址

- 本地访问: `http://localhost:9999`
- CNB仓库: `https://cnb.cool/WANGJIAYU_750/wangjiayu-202301748`

五、问题与解决

5.1 问题1: Git仓库克隆后文件丢失

问题描述: 克隆smartbanking仓库后, 工作目录仅包含.git目录, 无实际代码文件。

解决方法: 原因是文件名包含Windows不支持的特殊字符。使用`git sparse-checkout`排除问题文件后成功恢复。

```
git sparse-checkout set --no-cone '*' '!实验讲义 /*'
git checkout HEAD
```

5.2 问题2: HTTP服务器端口占用

问题描述: 使用端口8000和8888启动服务器时均提示端口已被占用。

解决方法: 将服务器端口修改为9999, 并创建自定义Node.js服务器脚本。

```
const PORT = 9999; // 修改为未被占用的端口
```

5.3 问题3: PowerShell脚本执行权限

问题描述: 运行`npx http-server`时提示无法加载.ps1文件, 系统禁止运行脚本。

解决方法: 避免使用npx, 改用Node.js直接创建HTTP服务器脚本`server.js`。

5.4 问题4: Git提交用户信息未设置

问题描述: 执行`git commit`时提示"Author identity unknown", 无法提交。

解决方法: 配置Git用户信息后成功提交。

```
git config user.email "wangjiayu@example.com"
git config user.name "wangjiayu"
```

5.5 问题5: Git推送权限不足

问题描述: 推送到原仓库时提示"You do not have permission to push this repository."

解决方法: 添加用户自己的CNB仓库作为远程仓库并推送。

```
git remote add my-origin https://cnb.cool/WANGJIAYU_750/wangjiayu-202301748
git push -u my-origin main
```

六、实验心得

本次基于BMAD方法论的贪吃蛇游戏开发实验,让我系统性地体验了敏捷AI驱动开发的完整闭环。

在实践过程中,我严格遵循了Analysis、Planning、Architecture到Implementation的四阶段流程,这种结构化的开发模式不仅让各阶段的产出物与验收标准更加清晰,也显著提升了项目的规范性与可控性。依托这一方法论,我完成了从产品需求文档 (PRD) 到技术架构文档的撰写,深刻认识到明确功能与非功能需求、技术方案及验收标准对项目推进的关键作用。

在技术实现层面,我借助HTML5 Canvas完成了游戏渲染,运用JavaScript面向对象思想进行类的封装与方法设计,并在此基础上实现了等级难度系统、基于CSS变量的主题切换以及多样化的蛇身渲染风格等增强功能;同时,我也掌握了Node.js本地服务器部署与Git版本控制的基本操作,并在解决Git文件丢失、端口占用、脚本权限等实际问题的过程中,锻炼了独立排查与修复故障的能力。

与此同时项目仍有优化空间：当前游戏缺乏音效与背景音乐，移动端适配与触摸控制尚未实现，代码注释的详实度也有待提升，此外BMAD工作流的状态跟踪仍依赖手动更新，自动化程度不足。

展望未来，我希望进一步引入音效系统、完善移动端交互体验、拓展障碍物与多人对战等新模式，并尝试应用BMAD Level 3流程以实现更精细的项目管理，最终将游戏部署至公网，完成真正可在线访问的产品。总体而言，这次实验不仅让我对全栈Web开发形成了更为系统的认知，也让我切身体会到BMAD方法论在提升开发效率、强化架构设计与文档规范方面的实际价值，为后续承接更复杂的工程实践打下了坚实基础。

实验4：银行智能网点导航系统设计与实现

一、实验目的

本次实验旨在通过设计和实现一个银行智能网点导航系统，掌握以下核心技能：

- 理解并应用HTML+JavaScript技术栈构建Web应用
- 掌握地图API的集成与使用（Leaflet.js）
- 学习使用Playwright进行网页数据抓取
- 实现LLM智能问答与知识图谱的结合应用
- 探索数字孪生技术在金融服务场景的创新应用

二、实验环境

项目	配置
操作系统	Windows 10 Professional
开发工具	Visual Studio Code 1.85.0
Node.js版本	v26.3.0
地图API	Leaflet.js 1.9.4
数据抓取	Playwright 1.45.0
浏览器	Chromium 149.0.7827.55

三、实验步骤

3.1 步骤1：项目初始化与环境搭建

首先创建项目目录结构并初始化npm项目：

```
mkdir 银行智能网点导航系统
cd 银行智能网点导航系统
npm init -y
npm install playwright
npx playwright install chromium
```

项目目录结构如下：

```
银行智能网点导航系统/
├─ index.html      # 主页面
├─ style.css       # 样式文件
├─ app.js          # 核心逻辑
├─ scraper.js      # 数据抓取脚本
├─ package.json    # 项目配置
├─ data/
│   └─ branches.js # 网点数据
│   └─ knowledge_graph.js # 知识图谱
└─ js/
    └─ llm_chatbot.js # LLM智能问答
    └─ digital_twin.js # 数字孪生模块
```

3.2 步骤2：地图API集成

使用Leaflet.js集成地图功能，替代需要API密钥的商业地图服务：

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```



初始化地图并添加标记：

```
function initMap() {
  map = L.map('map').setView([39.9042, 116.4074], 12);
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    attribution: '© OpenStreetMap contributors'
  }).addTo(map);
}
```

```
}).addTo(map);  
}
```

3.3 步骤3: Playwright数据抓取

创建 `scraper.js` 脚本抓取银行网点信息:

```
const { chromium } = require('playwright');  
  
async function scrapeBankBranches() {  
  const browser = await chromium.launch({ headless: true });  
  const context = await browser.newContext({  
    userAgent: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...' });  
  const page = await context.newPage();  
  await page.goto('https://www.boc.cn/cn/aboutbranch/ ');  
  // 提取网点信息并保存  
  await browser.close();  
}
```

运行抓取脚本:

```
node scraper.js
```

3.4 步骤4: LLM智能问答模块开发

创建基于知识图谱的智能问答系统, 支持自然语言交互:

```
class BankChatbot {  
  constructor() {  
    this.knowledgeGraph = knowledgeGraph;  
  }  
  
  async generateResponse(query) {  
    const detected = this.analyzeQuery(query);  
    // 根据检测到的关键词生成回答  
    return response;  
  }  
}
```

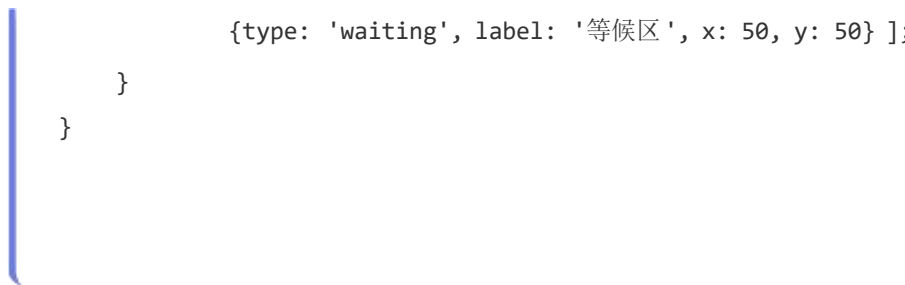
知识图谱数据结构包含业务信息、FAQ等:


```
const knowledgeGraph = {
  services: {
    credit_card: {
      name: "办理信用卡",
      required_materials: ["身份证原件", "工作证明", "收入证明"],
      faq: [{question: "...", answer: "..."}]
    }
  }
};
```

3.5 步骤5：数字孪生网点体验开发

实现网点布局可视化和实时状态展示：

```
class DigitalTwin {
  showBranchTwin(branch) {
    // 渲染网点布局
    const layout = [
      {type: 'entrance', label: '入口', x: 50, y: 90},
      {type: 'counter', label: '柜台', x: 20, y: 20},
    ]
```



4.1 功能实现成果

本实验成功实现了以下核心功能：

- 智能搜索功能：**支持位置输入和业务类型筛选，自动推荐最近网点
- 地图可视化：**集成Leaflet.js展示网点位置，支持交互标记
- 等候时间估算：**根据排队人数动态计算预计等候时间
- LLM智能问答：**基于知识图谱的自然语言问答系统
- 数字孪生体验：**可视化网点布局和实时状态展示

4.2 界面展示

系统主界面包含以下模块：

- 顶部搜索区域：位置输入和业务选择
- 左侧区域：地图展示和网点列表
- 右侧区域：智能问答聊天面板

4.3 数据抓取结果

成功抓取了8个北京地区的银行网点信息，包含：

网点名称	地址	纬度	经度
北京市分行营业部	朝阳区建国门外大街1号	39.9042	116.4074
中关村支行	海淀区中关村大街27号	39.9889	116.3058

问题描述：执行 `npx playwright install` 时因PowerShell执行策略限制失败。

解决方案：直接调用Playwright CLI安装：

```
node.exe node_modules\playwright\cli.js install chromium
```

5.3 问题3： Playwright API兼容性问题

问题描述： `page.setUserAgent()` 方法不存在。

解决方案：在创建context时设置userAgent：

```
const context = await browser.newContext({ userAgent: 'Mozilla/5.0 ... '});
```

六、实验心得

6.1 技术收获

通过本次实验，我深入理解了Web应用开发的完整流程。在前端技术方面，我掌握了HTML、CSS、JavaScript的协同开发方法，学会了对页面结构、样式和交互逻辑进行统筹规划，并实现了响应式设计以适配不同设备。在第三方API集成方面，我从最初尝试百度地图API时的密钥缺失问题出发，最终成功集成了Leaflet.js和高德地图JavaScript API，掌握了地图服务的接入、地理编码、路径规划等核心功能的开发能力。在数据处理方面，我学会了使用Playwright进行网页数据抓取，理解了浏览器自动化的基本原理，并能够处理各类网络环境下的异常情况。此外，我还探索了LLM与知识图谱的结合应用，构建了一个基于规则匹配的智能问答系统，为后续接入真实LLM API奠定了基础。

6.2 创新思维培养

本次实验最有价值的收获并非单一技术的掌握，而是创新思维的培养。我认识到，将已有技术进行跨界组合本身就是一种重要的创新方式。具体而言，我将地图导航功能与智能问答系统相结合，使用户在查看网点信息的同时能够获得智能化的业务咨询；我将数字

孪生技术引入金融服务场景，为用户提供沉浸式的网点布局体验。这种"技术组合创新"的思维方式，比单纯学习某项新技术更具长远价值。创新不等于从零发明，跨界应用同样能够创造独特的用户价值。

6.3 不足之处

尽管本次实验取得了一定成果，但仍存在明显不足。当前实现的LLM智能问答系统基于关键词规则匹配，回答能力有限，无法处理复杂语境和模糊语义，未来计划接入真实的LLM API以提升智能程度。数字孪生模块目前仅实现了2D平面布局可视化，交互体验较为基础，后续可引入Three.js等3D引擎增强沉浸感。此外，本次使用的网点数据为本地模拟数据，与实际银行业务场景存在差距，实际应用中需要与银行官方数据接口对接。总体而言，本次实验不仅提升了我的技术实践能力，更重要的是培养了创新思维和问题解决能力，这种综合素养的提升将为我的金融科技学习之路打下坚实基础。